

QAOA and Applications

QML

Contents

1	Overview	3
2	Combinatorial Optimization	3
2.1	The Setup	3
2.2	MaxCut	3
2.3	Other Combinatorial Problems	3
2.4	QUBO Formulation	4
3	The QAOA Algorithm	4
3.1	The Circuit	4
3.2	QAOA Circuit Diagram	5
3.3	The Cost Unitary	5
3.4	The Mixer Unitary	5
4	Why QAOA Works	6
4.1	Connection to Adiabatic Quantum Computing	6
4.2	QAOA Performance Guarantees	6
5	Parameter Optimization in QAOA	6
5.1	The Optimization Landscape	6
5.2	Optimization Strategies	7
5.3	Symmetries and Parameter Periodicity	7
6	QAOA Worked Example: MaxCut on a Triangle	7
7	Applications	8
7.1	Portfolio Optimization	8
7.2	Scheduling and Logistics	8
7.3	Machine Learning	9
8	QAOA Limitations and Open Questions	9
9	Warm-Starting QAOA	9

10	Beyond Standard QAOA	10
10.1	Multi-Angle QAOA (ma-QAOA)	10
10.2	ADAPT-QAOA	10
10.3	Recursive QAOA	10
11	Summary	10

1 Overview

the quantum approximate optimization algorithm (QAOA) is one of the most important variational quantum algorithms. its designed for combinatorial optimization problems — things like MaxCut, traveling salesman, portfolio optimization, scheduling. these problems r NP-hard classically, and while QAOA probably doesnt solve them in polynomial time, it might give better approximations than classical algorithms for certain instances.

QAOA is also important for QML bcs it shares the same variational framework as quantum classifiers. the circuit structure, parameter optimization, and trainability issues r all the same. understanding QAOA gives u a concrete example of how parameterized quantum circuits r used to solve real problems.

2 Combinatorial Optimization

2.1 The Setup

Definition 2.1. a combinatorial optimization problem asks: given a finite set of candidate solutions and an objective function, find the solution that maximizes (or minimizes) the objective.

formally: $\max_{z \in \{0,1\}^n} C(z)$ where C is the cost function and z is an n -bit binary string.

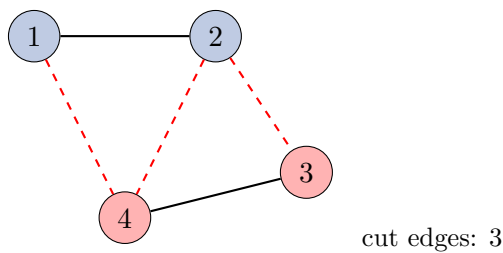
2.2 MaxCut

Definition 2.2. given a graph $G = (V, E)$, find a partition of the vertices into two sets that maximizes the number of edges between the sets (edges that are “cut”).

cost function:

$$C(z) = \sum_{(i,j) \in E} \frac{1 - z_i z_j}{2}$$

where $z_i \in \{-1, +1\}$ indicates which set vertex i belongs to.



maxcut is NP-hard in general. the best known classical approximation algorithm (Goemans-Williamson, 1995) achieves a ratio of ≈ 0.878 , and improving this is a major open problem (assuming the unique games conjecture).

2.3 Other Combinatorial Problems

- traveling salesman problem (TSP): find the shortest route visiting all cities exactly once
- graph coloring: color vertices such that no two adjacent vertices have the same color, using minimum colors

- portfolio optimization: maximize return while minimizing risk, subject to constraints
- job scheduling: assign jobs to machines to minimize total completion time

all of these can be formulated as QUBO (quadratic unconstrained binary optimization) problems.

2.4 QUBO Formulation

Definition 2.3. a QUBO (quadratic unconstrained binary optimization) problem:

$$\min_{x \in \{0,1\}^n} x^T Q x$$

where Q is an $n \times n$ upper triangular matrix encoding the objective function. diagonal elements encode linear terms, off-diagonal elements encode quadratic interactions.

Key Point. any QUBO can be mapped to an Ising hamiltonian by the substitution $x_i = (1 - z_i)/2$ where $z_i \in \{-1, +1\}$:

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j + \text{const}$$

this is a diagonal hamiltonian in the computational basis. its eigenvalues encode the cost function values. the ground state of H_C is the optimal solution.

Example 2.1. MaxCut on a triangle graph (3 vertices, 3 edges):

$$\text{cost function: } C(z) = \frac{1}{2}(3 - z_1 z_2 - z_2 z_3 - z_1 z_3)$$

ising hamiltonian:

$$H_C = -\frac{1}{2}(Z_1 Z_2 + Z_2 Z_3 + Z_1 Z_3) + \frac{3}{2}I$$

the ground state of $-H_C$ (or maximum of H_C) gives the optimal cut. by symmetry, the maximum cut is 2 (cut any 2 edges).

3 The QAOA Algorithm

3.1 The Circuit

QAOA was introduced by farhi, goldstone, and gutmann (2014). the algorithm uses a PQC with a specific alternating structure:

Definition 3.1. QAOA with p layers:

1. initialize: $|+\rangle^{\otimes n} = H^{\otimes n} |0\rangle^{\otimes n}$ (uniform superposition)
2. apply p layers of alternating unitaries:

$$|\gamma, \beta\rangle = \underbrace{e^{-i\beta_p B} e^{-i\gamma_p C}}_{\text{layer } p} \dots \underbrace{e^{-i\beta_1 B} e^{-i\gamma_1 C}}_{\text{layer 1}} |+\rangle^{\otimes n}$$

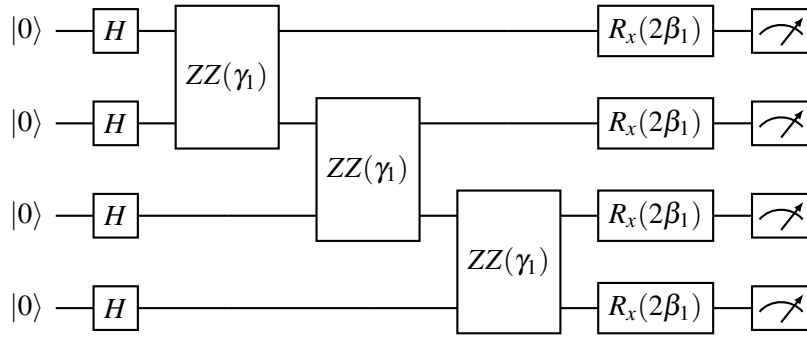
3. measure: compute $\langle C \rangle = \langle \gamma, \beta | C | \gamma, \beta \rangle$
4. optimize: use a classical optimizer to find γ^*, β^* that maximize $\langle C \rangle$

where:

- $C = H_C$ is the problem (cost) hamiltonian — diagonal in the computational basis, encodes the optimization objective
- $B = \sum_{i=1}^n X_i$ is the mixer hamiltonian — creates transitions between computational basis states
- $\gamma = (\gamma_1, \dots, \gamma_p)$ and $\beta = (\beta_1, \dots, \beta_p)$ r the variational parameters ($2p$ total)

3.2 QAOA Circuit Diagram

for MaxCut on a 4-vertex graph with $p = 1$:

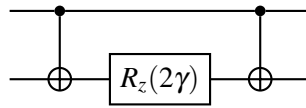


where $ZZ(\gamma) = e^{-i\gamma Z_i Z_j / 2}$ is the ZZ-interaction gate for each edge (i, j) in the graph.

3.3 The Cost Unitary

$$e^{-i\gamma C} = \prod_{(i,j) \in E} e^{-i\gamma Z_i Z_j / 2}$$

each ZZ-interaction can be decomposed into native gates:



this implements $e^{-i\gamma Z_i Z_j}$ (up to global phase) using two CNOTs and one R_z gate.

3.4 The Mixer Unitary

$$e^{-i\beta B} = e^{-i\beta \sum_j X_j} = \prod_j e^{-i\beta X_j} = \prod_j R_x(2\beta)$$

the mixer is just individual R_x rotations on each qubit — easy to implement.

4 Why QAOA Works

4.1 Connection to Adiabatic Quantum Computing

Intuition. QAOA can be understood as a trotterized version of adiabatic quantum computing. in adiabatic QC, u start with the ground state of an easy hamiltonian B (which is $|+\rangle^{\otimes n}$ for $B = \sum X_i$) and slowly evolve to the ground state of the problem hamiltonian C . the adiabatic theorem says if u go slowly enough, u stay in the ground state throughout.

QAOA approximates this continuous evolution with p discrete steps. as $p \rightarrow \infty$, QAOA can reproduce the adiabatic evolution exactly. for finite p , QAOA is a variational approximation — the optimizer finds the best discrete schedule.

Key Point. the relationship:

- $p = 1$: very coarse approximation. can still beat random guessing but typically far from optimal.
- $p = O(\text{poly}(n))$: can approximate adiabatic evolution. in principle, can find the optimal solution.
- $p \rightarrow \infty$: converges to the exact solution (under mild conditions).
- in practice on NISQ hardware: $p \sim 1\text{--}10$ due to noise constraints.

4.2 QAOA Performance Guarantees

Key Point. for MaxCut on 3-regular graphs (every vertex has degree 3), farhi et al. showed that QAOA with $p = 1$ achieves an approximation ratio of at least **0.6924** (i.e., cuts at least 69.24% of the maximum number of cuttable edges).

compare to:

- random assignment: 0.5 (50% of edges cut in expectation)
- greedy algorithm: can do better than random but no worst-case guarantee
- goemans-williamson SDP: 0.878 ratio
- QAOA $p = 1$: 0.6924
- QAOA with larger p : approaches 1.0 as p increases

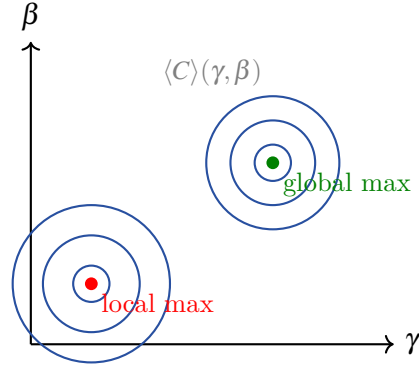
so $p = 1$ QAOA doesnt beat goemans-williamson. but for larger p , QAOA might outperform classical algorithms for certain instances. this is an active research question.

5 Parameter Optimization in QAOA

5.1 The Optimization Landscape

QAOA has $2p$ parameters: $\gamma_1, \dots, \gamma_p, \beta_1, \dots, \beta_p$.

for small p , the landscape is low-dimensional and can be explored relatively easily. for $p = 1$ (just 2 parameters), u can even plot the landscape:



5.2 Optimization Strategies

grid search (for $p = 1, 2$): systematically evaluate $\langle C \rangle$ on a grid of (γ, β) values. simple and works for low p .

classical optimizers: COBYLA, Nelder-Mead, L-BFGS-B for small p . gradient-based methods (using parameter shift rule) for larger p .

parameter initialization: random initialization can get stuck in local minima. strategies:

- linear ramp: set $\gamma_k = k\gamma_{\max}/p$ and $\beta_k = (1 - k/p)\beta_{\max}$ (mimics adiabatic schedule)
- INTERP: optimize for p layers, then interpolate to get initial parameters for $p + 1$ layers (layer-by-layer growth)
- transfer learning: optimize on a small instance, transfer optimal angles to a larger instance of the same problem type

Key Point. a nice property of QAOA: the optimal parameters for MaxCut often concentrate — they tend to be similar across different graph instances of the same type and size. this means u can “pre-train” on small graphs and transfer the parameters.

5.3 Symmetries and Parameter Periodicity

the QAOA cost function has symmetries that can be exploited:

- γ and β r periodic: $\gamma \in [0, 2\pi)$, $\beta \in [0, \pi)$
- for MaxCut: the landscape has additional symmetries that reduce the search space
- these symmetries can be used to restrict the optimization domain

6 QAOA Worked Example: MaxCut on a Triangle

Example 6.1. MaxCut on a triangle graph (3 vertices, 3 edges) with $p = 1$.

cost hamiltonian:

$$C = \frac{1}{2}(3I - Z_1Z_2 - Z_2Z_3 - Z_1Z_3)$$

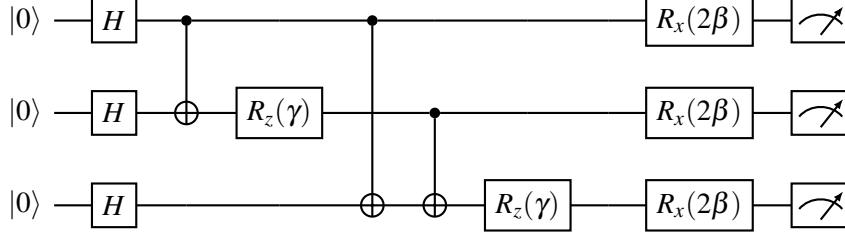
cost unitary:

$$e^{-i\gamma C} = e^{-i3\gamma/2} \cdot e^{i\gamma Z_1Z_2/2} \cdot e^{i\gamma Z_2Z_3/2} \cdot e^{i\gamma Z_1Z_3/2}$$

mixer unitary:

$$e^{-i\beta B} = R_x(2\beta)_1 \otimes R_x(2\beta)_2 \otimes R_x(2\beta)_3$$

circuit:



(simplified — the ZZ interactions share some CNOT gates and R_z rotations)

optimal parameters (computed analytically for this small case):

$$\gamma^* \approx 0.615, \beta^* \approx 0.393$$

expected cost: $\langle C \rangle \approx 2.0$ (the maximum cut is 2 edges).

the approximation ratio at $p = 1$ is quite good for this small instance.

7 Applications

7.1 Portfolio Optimization

Definition 7.1. given n assets with expected returns μ and covariance matrix Σ , find the portfolio weights w that maximize:

$$\text{maximize } \mu^T w - q \cdot w^T \Sigma w \quad \text{subject to } \sum_i w_i = B, \quad w_i \in \{0, 1\}$$

where q is a risk aversion parameter and B is the budget.

this is a QUBO problem: each binary variable w_i represents whether to include asset i in the portfolio. QAOA can be applied directly.

Intuition. portfolio optimization is one of the most commonly cited near-term applications of quantum computing. financial institutions (JPMorgan, Goldman Sachs, etc.) are actively exploring quantum approaches. the problem is NP-hard for integer constraints, and real portfolios can have hundreds of assets. whether QAOA provides practical advantages for realistic problem sizes is still an open question.

7.2 Scheduling and Logistics

- vehicle routing: assign routes to delivery vehicles to minimize total distance. maps to QUBO.
- job shop scheduling: assign jobs to machines to minimize makespan. maps to QUBO with constraints.
- network flow: optimize routing in communication or transportation networks.

7.3 Machine Learning

Key Point. QAOA connects to QML in several ways:

1. shared variational framework: QAOA and variational classifiers use the same optimize-measure-repeat loop
2. combinatorial optimization for ML: feature selection, hyperparameter optimization, and neural architecture search can be formulated as combinatorial problems solvable by QAOA
3. QAOA-inspired classifiers: the alternating cost/mixer structure can be adapted for classification tasks
4. graph-based ML: QAOA naturally operates on graph problems. graph neural networks (GNNs) solve similar problems classically. quantum GNNs combine both.

8 QAOA Limitations and Open Questions

Key Point. honest assessment of QAOA:

1. no proven advantage: for any fixed p , there's no proof that QAOA outperforms the best classical algorithms. the approximation ratio for $p = 1$ is below Goemans-Williamson.
2. noise sensitivity: QAOA circuits grow with problem size (more qubits, more ZZ interactions). noise limits practical problem sizes to $\sim 20\text{--}50$ variables on current hardware.
3. parameter optimization: for large p , the optimization landscape can have many local minima. barren plateaus can occur for deep QAOA circuits.
4. classical simulation: for low p , QAOA can often be simulated classically (the circuit doesn't generate enough entanglement). this weakens the case for quantum advantage.
5. problem encoding overhead: mapping real-world constraints to QUBO often requires auxiliary qubits and penalty terms, increasing the problem size significantly.

Intuition. my take on QAOA: it's a beautiful algorithm that elegantly connects quantum computing to combinatorial optimization. but the honest truth is that we don't yet know if it provides practical advantages over classical methods. the theoretical guarantees for small p are weaker than the best classical algorithms, and running large p on noisy hardware is currently impractical.

that said, QAOA is one of the best testbeds for near-term quantum computing. it's simple enough to implement on real hardware, its performance can be benchmarked against classical methods, and it might be the setting where we first see practical quantum advantage for optimization. the research is moving fast.

9 Warm-Starting QAOA

another interesting direction is warm-starting QAOA: instead of starting from the uniform superposition $|+\rangle^{\otimes n}$, start from a state that encodes a classical approximate solution.

Definition 9.1. warm-start QAOA: given a classical approximate solution z^* (e.g., from a greedy algorithm), initialize the quantum state to bias towards z^* :

$$|\psi_0\rangle = \bigotimes_{i=1}^n \left(\cos \frac{\alpha_i}{2} |0\rangle + \sin \frac{\alpha_i}{2} |1\rangle \right)$$

where α_i is chosen based on z_i^* . then run QAOA layers starting from this state.

the idea: let classical algorithms do the heavy lifting to get a good starting point, then use QAOA to improve it further. this hybrid approach is more practical than starting from scratch.

10 Beyond Standard QAOA

10.1 Multi-Angle QAOA (ma-QAOA)

instead of using the same γ for all problem terms and the same β for all mixer terms, use different angles for each term:

$$e^{-i\gamma C} \rightarrow \prod_{(i,j) \in E} e^{-i\gamma_{ij} Z_i Z_j}$$

more parameters ($|E| + n$ per layer instead of 2) but potentially better performance with fewer layers.

10.2 ADAPT-QAOA

iteratively grow the circuit by choosing the most impactful gate at each step (analogous to ADAPT-VQE in quantum chemistry). avoids the fixed ansatz structure and builds a problem-specific circuit.

10.3 Recursive QAOA

run QAOA on the full problem, measure one variable, fix it, and recurse on the remaining smaller problem. this can give better solutions than running QAOA once on the full problem.

11 Summary

Key Point. QAOA key points:

1. QAOA is a variational algorithm for combinatorial optimization
2. it alternates between cost ($e^{-i\gamma C}$) and mixer ($e^{-i\beta B}$) unitaries
3. connected to adiabatic quantum computing (trotterized annealing)
4. $2p$ parameters optimized classically
5. applications: MaxCut, portfolio optimization, scheduling, logistics
6. no proven quantum advantage yet for fixed p

7. shares the same variational framework as QML classifiers
8. variants (ma-QAOA, ADAPT-QAOA, warm-start) try to improve practical performance
9. the central challenge: can QAOA outperform the best classical algorithms for practically relevant problem sizes? open question.

next: from optimization to classification. variational quantum classifiers take the same PQC framework and apply it to supervised learning.